

Relatório 8 - Vídeo e Leitura: WebSocket para Dashboards (III)

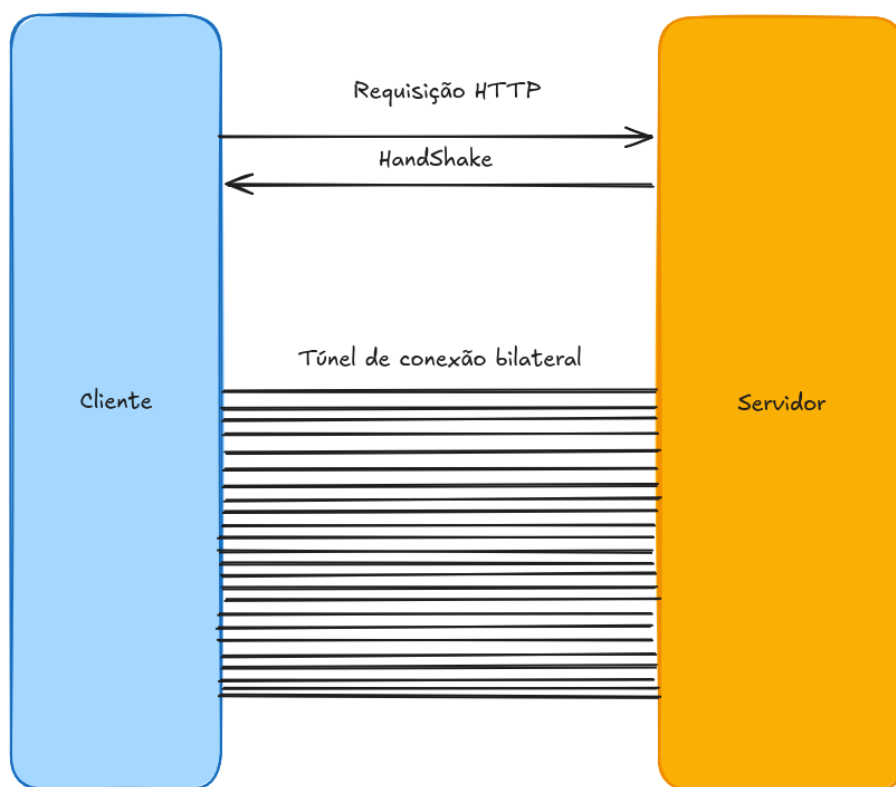
José Carlos Seben de Souza Leite

Descrição da atividade

Para começarmos esse card nos é apresentado o que são WebSockets que no caso são uma “melhoria situacional” do HTTP. Diferente do HTTP que trabalha com requisições únicas e encerra a comunicação com o servidor ao receber a resposta da requisição o WebSocket estabelece uma conexão continua que como nos foi apresentado no vídeo **“WebSocket // Programmer's Dictionary”** com o exemplo de um chat se fosse utilizando HTTP quando o lado A estivesse digitando seria aberto uma conexão com o servidor e ao enviar a mensagem feito um POST já para o lado B que vai receber a mensagem o cliente precisa ficar fazendo várias requisições ao servidor até receber a mensagem então cada requisição abrindo uma nova comunicação com o servidor, utilizando WebSocket esse problema seria resolvido pela sua comunicação continua.

Mas nem tudo são flores, utilizar o WebSocket em uma situação impropria apenas adicionaria uma camada de complexidade desnecessária a mais, além de provavelmente gerar um impacto direto do desempenho, então assim como tudo no ramo da tecnologia ele é algo situacional que cabe a nos identificar e o aplicar nos cenários que ele melhor desempenha.

Indo para o segundo vídeo do card **“WebSockets - Live de Python #164”** entramos em uma aplicação mais prática do websocket, mas com uma contextualização geral de história do http, mas indo para a parte que realmente importa entramos na comparação de http e websocket que não foge muito do que já descrevi na primeira parte do relatório, mas aqui com um complemento de como é estabelecido a conexão, inicialmente em uma “troca de mensagens” entre o cliente e o servidor como no protocolo HTTP, mas após essa comunicação ser estabelecida abre-se um “túnel” de comunicação continuo e bilateral entre servidor e cliente, sem ser diversas conexões diferentes abertas uma a uma por requisição.



Na representação a cima mostra como é estabelecido a conexão do WebSock primeiro com a comunicação http entre cliente e servidor que é chamado de handshake e após isso é estabelecido a conexão continua bilateral do websocket.

Se tratando da documentação "**WebSockets - FastAPI**" ele é sobre exemplos práticos de o que podemos fazer e como podemos fazer como no exemplo de por estarmos trabalhando com websockets não utilizarmos HTTPException e sim WebSocketException além desses temos mais alguns pontos importantes como.

- **Injeção de Dependências:** O FastAPI permite reutilizar o sistema de Depends para validar parâmetros de consulta.
- **Segurança Antecipada:** É possível realizar a autenticação do usuário logo na abertura da conexão, garantindo que o "túnel" só seja estabelecido para clientes autorizados.
- **Ciclo de Vida e Desconexão:** A documentação enfatiza o uso do WebSocketDisconnect para capturar quando um cliente encerra a sessão, permitindo que o servidor limpe recursos ou avise outros usuários.

Dificuldades

Conclusões

Podemos concluir que, como praticamente toda ferramenta, o uso de WebSockets é situacional. Ele não será aplicado em todos os cenários, mas é ideal para situações que exigem conexões contínuas para reduzir o número de chamadas. Diferente do HTTP, onde o receptor precisaria realizar diversas requisições para verificar se uma mensagem chegou, o WebSocket permite o recebimento imediato assim que a resposta está disponível.

Referencias

Videos:

▶ WebSocket // Dicionário do Programador

▶ WebSockets - Live de Python #164

Docs:

<https://fastapi.tiangolo.com/advanced/websockets/>